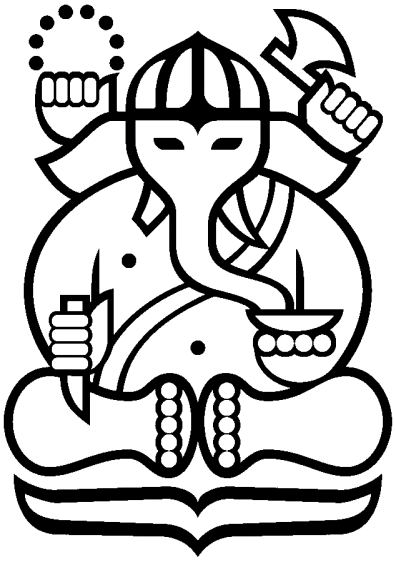
TUGAS BESAR 2 IF3070 DASAR INTELEGENSI ARTIFISIAL
Implementasi Algoritma Pembelajaran Mesin



Kelompok 22
Anggota Kelompok :

K01 Dinda Thalia Fahira	18222055@std.stei.itb.ac.id
K03 Mohamad Maulana Firdaus R	18222140@std.stei.itb.ac.id

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2024

DAFTAR ISI

DAFTAR ISI **2**
LATAR BELAKANG **3**
DASAR TEORI **4**
K-Nearest Neighbors 4
Gaussian Naive-Bayes 5
DATA INFORMATION **6**
EXPLORATORY DATA ANALYSIS **9**
DATA PREPARATION **12**
Handling Missing Values 12
Feature Dropper 12
Feature Transformation 12
MODEL IMPLEMENTATION **13**
K-Nearest Neighbors 13
Gaussian Naive Bayes 15
HASIL DAN ANALISIS **18**
Perbandingan hasil scikit-learn dan KNN 18
Perbandingan hasil hasil scikit-learn dan Naive-Bayes 19
Perbandingan hasil hasil KNN dan GNB 19
HYPERPARAMETER TUNING **20**
Perbandingan hasil KNN Base Model vs Tuned 20
Perbandingan hasil GNB Base Model vs Tuned 21
ERROR ANALYSIS **22**
KNN Tuned Model 22
GNB Tuned Model 23
PEMBAGIAN TUGAS **24**
REFERENSI **25**

LATAR BELAKANG

Tugas ini bertujuan untuk memberikan pemahaman kepada mahasiswa tentang implementasi algoritma machine learning dengan mengaplikasikan dua algoritma yang telah dipelajari di kelas, yaitu K-Nearest Neighbors (KNN) dan Gaussian Naive-Bayes (GNB), pada dataset PhiUSIIL Phising URL Dataset. Implementasi ini dilakukan dengan pendekatan from scratch, sehingga mahasiswa dapat memahami logika dasar algoritma.

Dalam tugas ini, mahasiswa diminta untuk mengembangkan implementasi algoritma KNN dan Gaussian Naive-Bayes. Untuk algoritma KNN, implementasi harus dapat menerima setidaknya dua parameter masukan, yaitu jumlah tetangga dan metrik jarak antar data, dengan pilihan metrik seperti Euclidean, Manhattan, dan Minkowski. Sedangkan untuk Gaussian Naive-Bayes, implementasi harus mencakup seluruh langkah perhitungan probabilitas berdasarkan distribusi Gaussian. Selain itu, hasil implementasi algoritma harus dibandingkan dengan versi library scikit-learn untuk mengevaluasi performa.

Selain implementasi algoritma, model yang dibuat harus memiliki kemampuan untuk disimpan (save) dan dimuat ulang (load) agar dapat digunakan kembali. Sebagai tahap akhir, mahasiswa juga diharapkan untuk membuat submission ke Kaggle, yang memungkinkan evaluasi performa model secara eksternal. Dalam tugas ini, implementasi from scratch dapat dilakukan dalam bentuk kelas (class), seperti KNN atau GaussianNB, dan hanya diperbolehkan menggunakan library matematika dasar seperti numpy.

DASAR TEORI

K-Nearest Neighbors

K-Nearest Neighbors merupakan algoritma yang tidak memerlukan asumsi apa pun tentang distribusi data yang mendasarinya. Algoritma ini dapat menangani data numerik dan kategorikal, sehingga menjadi pilihan yang fleksibel untuk melakukan klasifikasi dan regresi. Algoritma ini adalah metode nonparametrik yang membuat prediksi berdasarkan kesamaan titik data dalam kumpulan data tertentu. K-NN sendiri kurang sensitif terhadap *outlier* dibandingkan dengan algoritma lainnya.

Algoritma K-NN bekerja dengan menemukan K tetangga terdekat ke titik data tertentu berdasarkan matrik jarak. Kelas atau nilai titik data kemudian ditentukan oleh suara rata-rata K tetangga. Pendekatan ini memungkinkan algoritma untuk beradaptasi dengan pola yang berbeda dan membuat prediksi berdasarkan struktur *local data*.

Berikut merupakan metrik jarak yang digunakan dalam algoritma KNN:

1. Euclidean Distance

$$distance(x,\ Xi)\ =\ \sqrt{\sum_{j=1}^{d}\left(x_j - X_{i_j}\right)^2}$$

2. Manhattan Distance

$$d(x,y)\ =\ \sum_{i=1}^{n}\left|x_i - y_i\right|$$

3. Minkowski Distance

$$d(x,y)\ =\ \left(\sum_{i=1}^{n}\left(x_i - y_i\right)^p\right)^{\frac{1}{p}}$$

Gaussian Naive-Bayes

Gaussian Naive Bayes adalah algoritma klasifikasi yang didasarkan pada teorema Bayes, yaitu metode statistik untuk menghitung probabilitas suatu kejadian berdasarkan serangkaian kondisi. Dalam Gaussian Naive Bayes, asumsi naif dibuat bahwa fitur-fitur data bersifat independen satu sama lain, yang akan menyederhanakan perhitungan.

Teorema Bayes

Misalkan kita mempunyai Hipotesis (H) dan bukti (E), Menurut teorema Bayes, hubungan antara probabilitas Hipotesis sebelum memperoleh bukti yang direpresentasikan sebagai P(H) dan probabilitas hipotesis setelah memperoleh bukti yang direpresentasikan sebagai P(H|E) adalah:

$$P(H|E) = \frac{P(E|H) \times P(H)}{P(E)}$$

Dengan:

- Probabilitas prior = P(H) adalah probabilitas sebelum mendapatkan bukti
- Probabilitas evidence = P(E) adalah probabilitas bukti dari kelas E
- Probabilitas posterior = P(H|E) adalah probabilitas setelah mendapatkan bukti
- Probabilitas likelihood = P(E|H) adalah probabilitas data E diberikan kelas H

Berikut merupakan langkah-langkah pada Gaussian Naive Bayes:

1. Menghitung probabilitas prior P(H) untuk setiap kelas dari data pelatihan.
2. Menghitung parameter distribusi Gaussian (mean dan varians) untuk setiap fitur dan kelas.
3. Menggunakan probabilitas likelihood P(E|H) berdasarkan fungi densitas probabilitas Gaussian:

$$P(E|H) = \frac{1}{\sqrt{2\pi\sigma^2}} \times exp\left(-\frac{(X-\mu)^2}{2\sigma^2}\right)$$

4. Menggabungkan prior dan likelihood untuk menghitung probabilitas posterior, lalu memilih kelas dengan probabilitas posterior tertinggi sebagai prediksi.

GNB memiliki kelebihan berupa efisiensi komputasi dan keandalan untuk dataset dengan asumsi independensi fitur. Namun, jika asumsi ini dilanggar, performa model bisa menurun.

DATA INFORMATION

PhiUSIIL Phishing URL (Website) merupakan dataset yang dirancang untuk analisis dan pendeteksian URL phising. Dataset ini berisi informasi teknis, karakteristik keamanan, dan deskripsi berbagai URL, yang dapat digunakan untuk melatih algoritma pembelajaran mesin dalam mendeteksi aktivitas phising.

Dataset ini terdiri dari 140,404 baris dan 56 kolom, yang meliputi berbagai fitur teknis, deskriptif, dan numerik. Berikut adalah deskripsi masing-masing kolom:

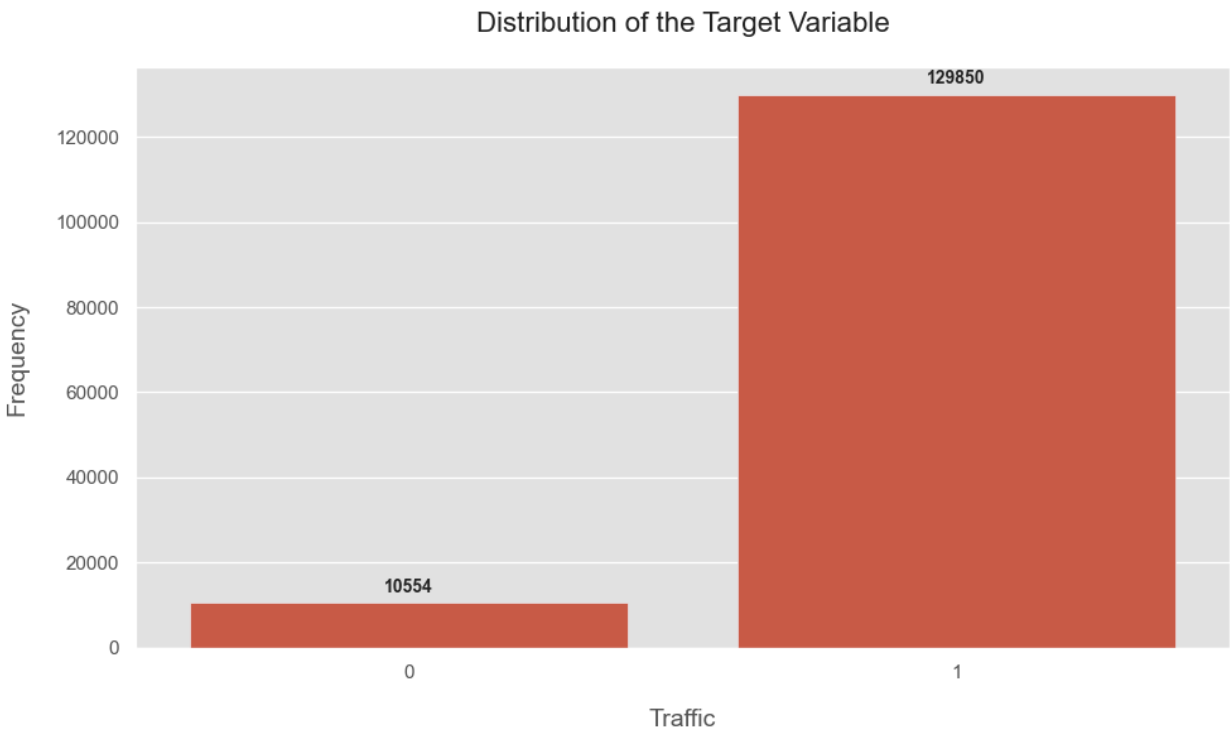
Tabel 3.1 Deskripsi kolom pada dataset

Kolom	Deskripsi
id	Pengidentifikasi unik untuk setiap baris data
FILENAME	Nama file yang terkait dengan URL
URL	URL lengkap yang dianalisis
URLLength	Panjang URL dalam karakter
Domain	Domain dari URL
DomainLength	Panjang domain dalam karakter
IsDomainIP	Apakah domain adalah alamat IP (1: ya, 0: tidak)
TLD	Top-Level Domain (contoh: .com, .org)
CharContinuationRate	Rasio kelanjutan karakter dalam URL, menggambarkan pola karakter yang berlanjut
TLDLegitimateProb	Probabilitas legitimasi TLD berdasarkan informasi historis
URLCharProb	Probabilitas karakter dalam URL dibandingkan karakter standar
TLDLength	Panjang TLD dalam karakter
NoOfSubDomain	Jumlah subdomain dalam URL
HasObfuscation	Apakah URL memiliki teknik obfuscation seperti masking atau encoding (1: ya, 0: tidak)
NoOfObfuscatedChar	Jumlah karakter yang menggunakan teknik obfuscation
ObfuscationRatio	Rasio obfuscation terhadap panjang URL
NoOfLettersInURL	Jumlah huruf dalam URL
LetterRatioInURL	Rasio huruf terhadap panjang URL

NoOfDegitsInURL	Jumlah angka dalam URL
DegitRatioInURL	Rasio angka terhadap panjang URL
NoOfEqualsInURL	Jumlah tanda sama dengan (=) dalam URL
NoOfQMarkInURL	Jumlah tanda tanya (?) dalam URL
NoOfAmpersandInURL	Jumlah tanda ampersand (&) dalam URL
NoOfOtherSpecialCharsInURL	Jumlah karakter khusus lainnya dalam URL
SpacialCharRatioInURL	Rasio karakter khusus terhadap panjang URL
IsHTTPS	Apakah URL menggunakan protokol HTTPS (1: ya, 0: tidak)
LineOfCode	Jumlah baris kode pada halaman web
LargestLineLength	Panjang baris terpanjang dalam halaman web
HasTitle	Apakah halaman web memiliki judul (1: ya, 0: tidak)
Title	Judul halaman web jika ada
DomainTitleMatchScore	Skor kesesuaian antara domain dan judul halaman web
URLTitleMatchScore	Skor kesesuaian antara URL dan judul halaman web
HasFavicon	Apakah halaman web memiliki favicon (1: ya, 0: tidak)
Robots	Informasi dari file robots.txt
IsResponsive	Apakah halaman web bersifat responsif (1: ya, 0: tidak)
NoOfURLRedirect	Jumlah redirect yang dilakukan oleh halaman web
NoOfSelfRedirect	Jumlah redirect internal pada halaman web
HasDescription	Apakah halaman web memiliki deskripsi meta (1: ya, 0: tidak)
NoOfPopup	Jumlah pop-up yang ada pada halaman web
NoOfiFrame	Jumlah elemen iFrame pada halaman web
HasExternalFormSubmit	Apakah halaman web memiliki form yang mengirim data ke domain eksternal (1: ya, 0: tidak)
HasSocialNet	Apakah halaman web memiliki elemen yang terhubung ke jejaring sosial (1: ya, 0: tidak)
HasSubmitButton	Apakah halaman web memiliki tombol submit (1: ya, 0: tidak)
HasHiddenFields	Apakah halaman web memiliki elemen input tersembunyi (1: ya, 0:

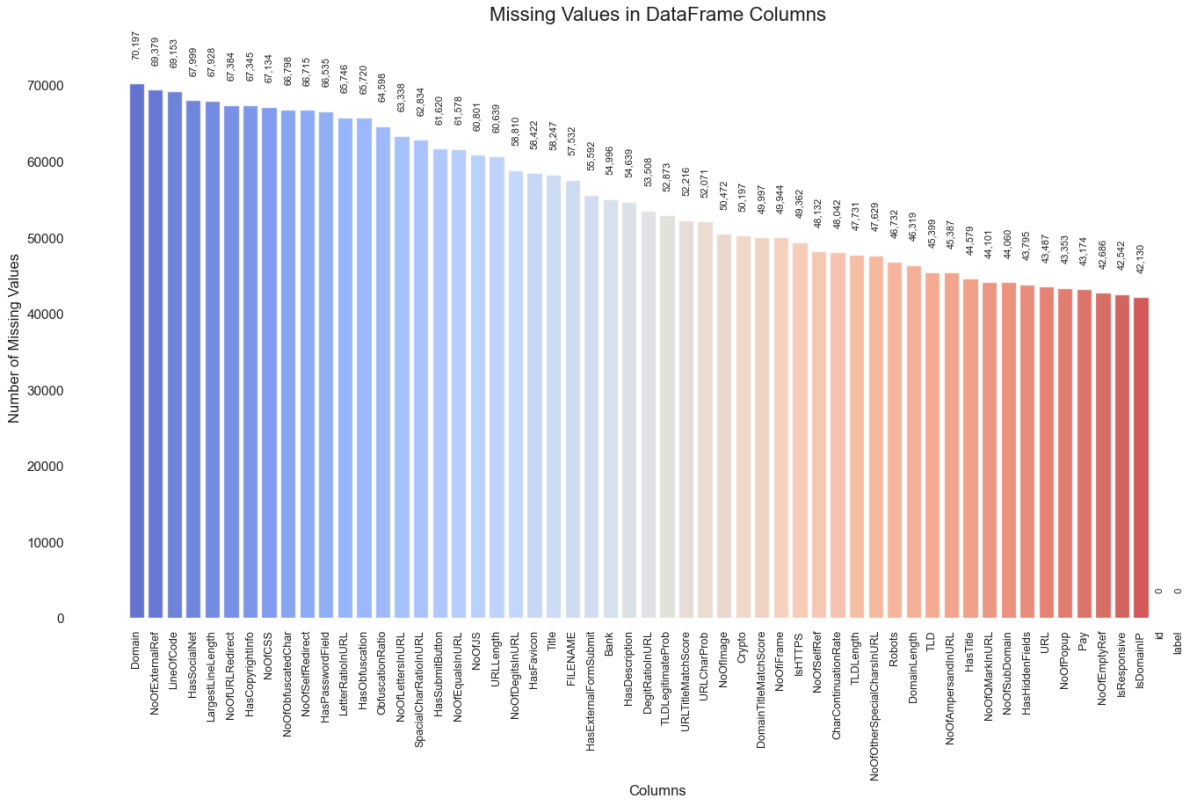
	tidak)
HasPasswordField	Apakah halaman web memiliki elemen input untuk password (1: ya, 0: tidak)
Bank	Indikator terkait elemen perbankan dalam halaman web
Pay	Indikator terkait elemen pembayaran dalam halaman web
Crypto	Indikator terkait elemen cryptocurrency dalam halaman web
HasCopyrightInfo	Apakah halaman web memiliki informasi hak cipta (1: ya, 0: tidak)
NoOfImage	Jumlah gambar pada halaman web
NoOfCSS	Jumlah file CSS pada halaman web
NoOfJS	Jumlah file JavaScript pada halaman web
NoOfSelfRef	Jumlah referensi internal dalam halaman web
NoOfEmptyRef	Jumlah referensi kosong dalam halaman web
NoOfExternalRef	Jumlah referensi eksternal dalam halaman web
label	Label target untuk klasifikasi (0: phishing, 1: bukan phishing)

EXPLORATORY DATA ANALYSIS



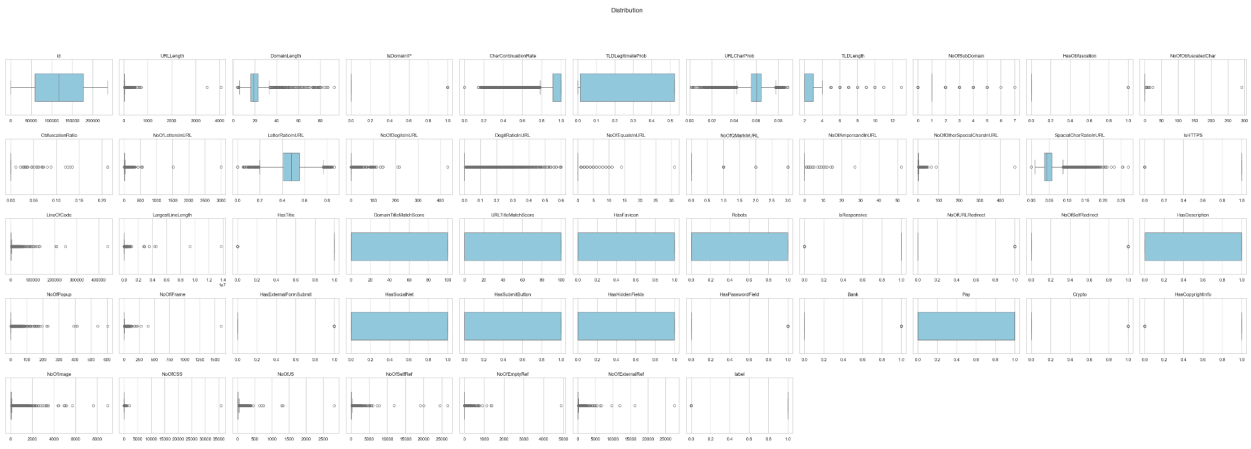
Gambar 4.1 Diagram analisis *distribution of the target variable*

Hasil dari diagram diatas menunjukkan adanya ketidakseimbangan pada distribusi variabel target, di mana **non phishing URL** jauh lebih dominan dibandingkan **phishing URL**. Ketidakseimbangan ini berpotensi mempengaruhi performa model klasifikasi, sehingga diperlukan penanganan khusus untuk memastikan model dapat memberikan prediksi yang akurat dan seimbang pada kedua target.



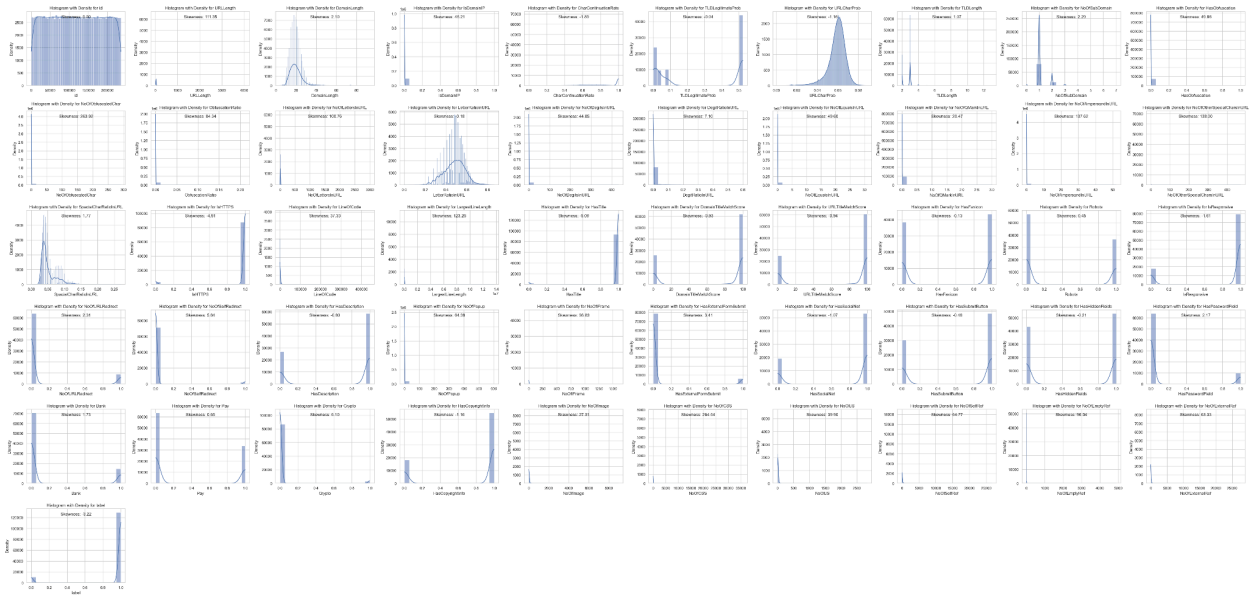
Gambar 4.2 Diagram analisis *missing values in DataFrame columns*

Dari hasil analisis missing values diatas, terlihat bahwa beberapa kolom dalam dataset memiliki jumlah nilai hilang yang cukup signifikan. Kolom dengan jumlah nilai hilang tertinggi adalah **Domain** dengan 70,197 nilai hilang, diikuti oleh kolom-kolom lain. Sementara itu, kolom **Id** dan **label** tidak memiliki nilai hilang sama sekali. Dengan begitu perlu dilakukannya *handling missing values.*



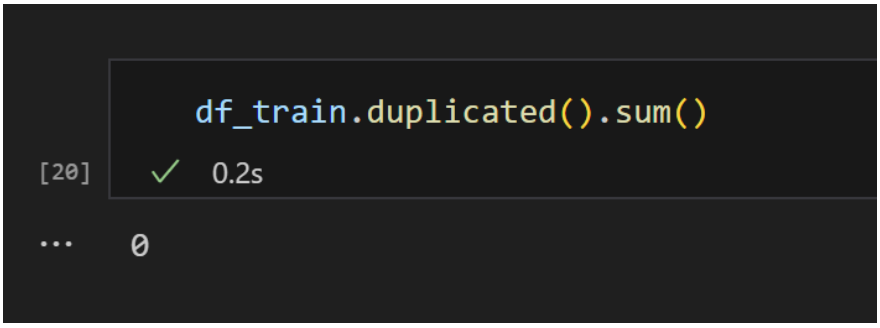
Gambar 4.3 Analisis *outliers*

Visualisasi distribusi pada fitur numerik menunjukkan bahwa banyak fitur memiliki rentang nilai yang luas dengan keberadaan outlier signifikan.



Gambar 4.4 histogram distribusi fitur numerik

Sebagian besar fitur numerik pada data ini menunjukkan distribusi dengan *skewness* tinggi. Hal ini menunjukkan adanya banyak *sparsity* dalam dataset.



Gambar 4.5 Jumlah data duplikat

Diketahui dalam data ini tidak memiliki data duplikat sehingga seluruh data sudah aman tanpa adanya kemungkinan bias yang muncul dari data ganda.

DATA PREPARATION

Dalam melakukan proses machine learning tentunya kita perlu mempersiapkan datanya dahulu agar compatible dengan model itu sendiri dan juga untuk memastikan kualitas datanya. Berikut ini merupakan tahapan yang digunakan dalam mempersiapkan data.

Handling Missing Values

Null values ditangani menggunakan dua metode utama:

a. Rule-Based Imputation: Digunakan ketika nilai kosong dapat diisi berdasarkan informasi dari kolom lain. Misalnya, nilai kolom isHTTPS dapat ditentukan dengan melihat apakah URL mengandung "https" atau tidak.
b. Simple Imputer (Most Frequent): Metode ini digunakan dengan asumsi bahwa nilai yang paling sering muncul akan muncul kembali, sehingga cocok untuk mengisi nilai kosong pada kolom dengan distribusi frekuensi yang jelas.

Metode lain tidak digunakan karena alasan berikut:

- Mean/Median Imputation: Tidak dipilih karena data memiliki distribusi yang skewed dan juga null yang banyak, sehingga nilai mean atau median tidak representatif.
- KNN Imputer: Tidak relevan karena antar baris tidak memiliki hubungan kuat, seperti yang biasanya terjadi pada data time series atau spasial.
- Iterative Imputer: Dihindari karena hubungan antar kolom dalam dataset ini cenderung lemah atau redundant, terutama karena banyak kolom merupakan turunan dari kolom utama seperti URL atau domain.

Feature Dropper

Kolom tertentu, seperti url, domain, filename, id, dan tld, dihapus karena memiliki nilai null yang terlalu banyak dan tidak dapat diisi dengan metode yang memadai. Selain itu juga domain, filename dan url memiliki diversitas yang sangat tinggi yang mana tentunya tidak bermanfaat karena tidak akan memberikan informasi apa apa.

Feature Transformation

Transformasi logaritmik dilakukan pada fitur dengan jumlah nilai unik lebih dari 50 untuk mengurangi skewness dan menangani outliers. Outlier disini dihandle dengan transformasi ini, karena diketahui bahwa outliers dalam data ini bukan merupakan sebuah kerusakan data, melainkan sebuah informasi. Misalnya dalam DomainLength terdapat nilai 1000, maka hal ini dapat divalidasi dan dibenarkan karena kita bisa langsung mengecek lengthnya. Jadi demi mengurangi dampak dari outlier ini dilakukan transformation. Selain itu juga transformation ini penting dalam mengatasi data yang skewed agar lebih normal.

MODEL IMPLEMENTATION

K-Nearest Neighbors

Berikut merupakan implementasi dari KNN dengan menggunakan python yang dioptimalkan dengan menggunakan threading.



```python
import numpy as np
import pandas as pd
from collections import Counter
from concurrent.futures import ThreadPoolExecutor

class KNN:
    def __init__(self, n_neighbor=3, metric='euclidean', p=3, n_jobs=-1):
        self.n_neighbor = n_neighbor
        self.metric = metric
        self.p = p
        self.n_jobs = n_jobs
        self.X_train = None
        self.Y_train = None

    def _validate_and_convert(self, X, Y=None):
        """
        Validates and converts the input data (X and Y) to a compatible format.
        Ensures X is numeric and Y is of valid type and shape.
        """
        # Validate X
        if not isinstance(X, (pd.DataFrame, np.ndarray)):
            raise ValueError("X must be a Pandas DataFrame or a NumPy array.")
        if isinstance(X, pd.DataFrame):
            X = X.copy()
            X = X.apply(pd.to_numeric, errors='coerce')
        X = np.array(X, dtype=np.float64)

        if np.any(np.isnan(X)):
            raise ValueError("X contains NaN values. Handle missing values before passing the data.")

        # Validate Y if provided
        if Y is not None:
            if not isinstance(Y, (pd.Series, np.ndarray, list)):
                raise ValueError("Y must be a Pandas Series, NumPy array, or list.")
            Y = np.array(Y, dtype=np.float64)

            if len(Y) != len(X):
                raise ValueError("X and Y must have the same number of samples.")
            if np.any(pd.isnull(Y)):
                raise ValueError("Y contains missing values.")

        return X, Y

    def fit(self, X_train, Y_train):
        """
        Fits the KNN model with the training data.
        Validates and converts the input data.
        """
        X_train, Y_train = self._validate_and_convert(X_train, Y_train)
        self.X_train = X_train
        self.Y_train = Y_train

    def _compute_distance(self, x, X_train):
        """
        Computes the distance between a test sample and all training samples.
        Supports Euclidean, Manhattan, and Minkowski distances.
        """
        if self.metric == 'euclidean':
            return np.sqrt(((x - X_train) ** 2).sum(axis=1))
        elif self.metric == 'manhattan':
            return np.abs(x - X_train).sum(axis=1)
        elif self.metric == 'minkowski':
            return np.power(np.abs(x - X_train) ** self.p, 1 / self.p).sum(axis=1)
        else:
            raise ValueError(f"Unsupported metric '{self.metric}'")

    def _most_common_label(self, labels):
        """
        Determines the most common label among the k-nearest neighbors.
        """
        return Counter(labels).most_common(1)[0][0]

    def _predict_one(self, x):
        """
        Predicts the label for a single test sample.
        """
        distances = self._compute_distance(x, self.X_train)
        k_indices = np.argsort(distances)[:self.n_neighbor]
        k_nearest_labels = self.Y_train[k_indices]
        return self._most_common_label(k_nearest_labels)

    def predict(self, X_test):
        """
        Predicts the labels for all test samples.
        Validates and converts the test data.
        Uses parallel processing for efficiency.
        """
        if self.X_train is None or self.Y_train is None:
            raise ValueError("Model has not been fitted. Call 'fit' with training data first.")

        X_test, _ = self._validate_and_convert(X_test)
        with ThreadPoolExecutor(max_workers=self.n_jobs if self.n_jobs > 0 else None) as executor:
            predictions = list(executor.map(self._predict_one, X_test))

        return predictions
```

Gambar 6.1 Model implementation K-Nearest Neighbors

Algoritma KNN ini diimplementasikan sebagai sebuah class dimana di dalam class ini terdapat 7 buah fungsi, yaitu.

1. __init__

   Fungsi ini bertugas untuk melakukan konstruksi parameter terhadap algoritma, dimana beberapa parameter utama seperti jumlah tetangga terdekat (n_neighbor), metrik jarak (metric), parameter Minkowski (p), dan jumlah pekerjaan paralel (n_jobs) diinisialisasi baik sesuai parameter default ataupun sesuai input user.

2. _validate_and_convert

   Fungsi ini akan melakukan proses validasi input untuk memastikan bahwa data yang dimasukan dalam algoritma ini dapat diproses. Jika input bukan berupa Dataframe ataupun numpy array maka akan memberikan error. Jika input berupa DataFrame, semua kolom akan dikonversi menjadi numerik. Jika konversi gagal atau terdapat nilai kosong, fungsi ini akan memberikan error. Selain itu, fungsi ini juga memastikan label memiliki format yang sesuai dan tidak mengandung nilai kosong.

3. fit

   Fungsi ini akan melakukan penyimpanan data pelatihan (X_train dan Y_train) setelah melalui validasi dan konversi menggunakan _validate_and_convert. Data yang valid kemudian disimpan sebagai atribut internal kelas untuk digunakan dalam prediksi.

4. _compute_distance

   Fungsi ini bertanggung jawab untuk menghitung jarak antara satu sampel uji dengan seluruh sampel pelatihan berdasarkan metrik jarak yang dipilih. Metrik yang didukung meliputi Euclidean, Manhattan, dan Minkowski.

5. _most_common_label

   Fungsi ini menentukan label yang paling sering muncul di antara tetangga terdekat. Dengan menggunakan Counter, fungsi ini menghitung jumlah kemunculan setiap label dan mengembalikan label mayoritas sebagai hasil prediksi. Fungsi ini memastikan bahwa hasil prediksi didasarkan pada konsensus tetangga terdekat.

6. _predict_one

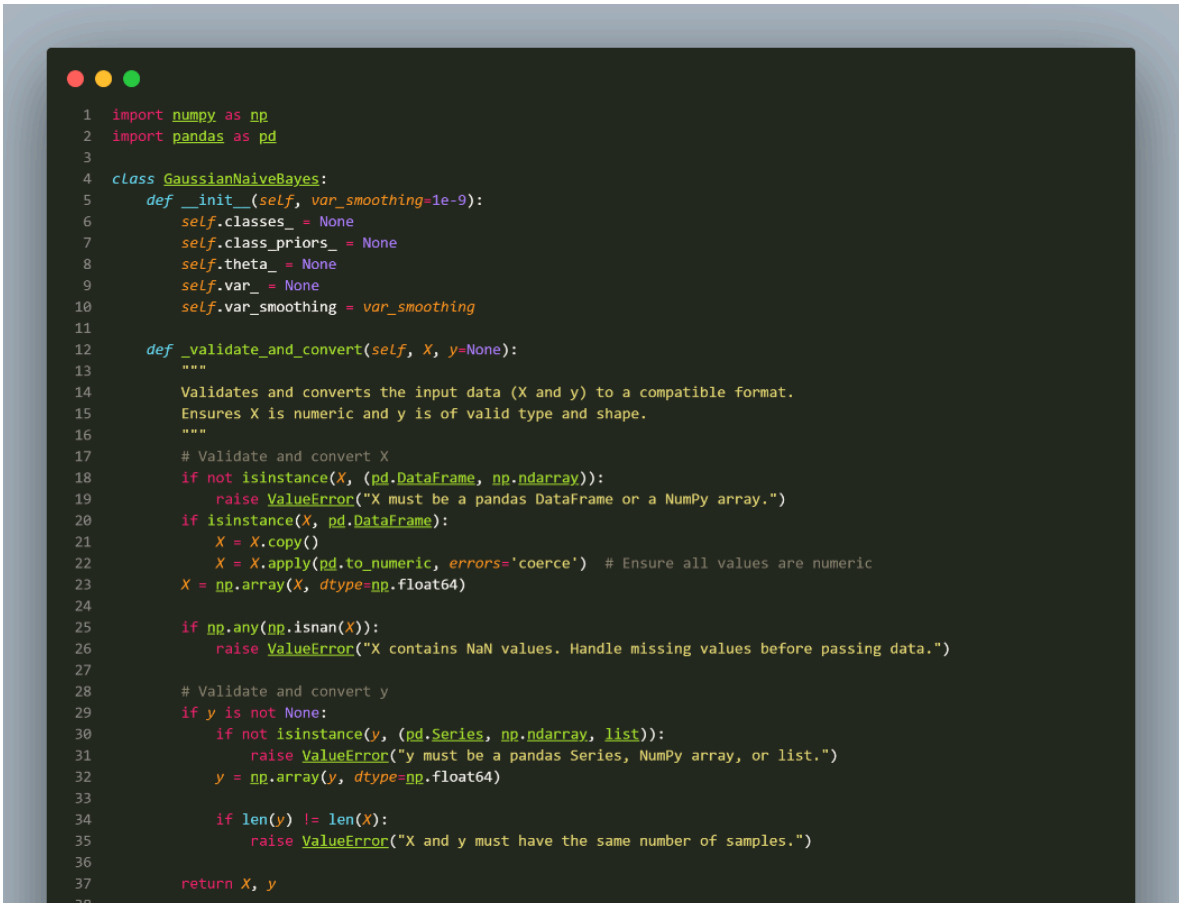
   Fungsi ini memprediksi label untuk satu sampel uji. Prosesnya melibatkan penghitungan jarak ke seluruh sampel pelatihan, pengambilan indeks k tetangga terdekat, dan penentuan label mayoritas menggunakan _most_common_label. Fungsi ini dirancang untuk modularitas sehingga dapat digunakan dalam proses prediksi batch.

7. predict

   Fungsi ini digunakan untuk memprediksi label dari dataset uji. Sebelum memulai prediksi, fungsi memastikan bahwa model telah dilatih dan data uji valid. Dengan bantuan ThreadPoolExecutor, fungsi ini melakukan prediksi paralel untuk meningkatkan efisiensi, terutama pada dataset besar. Fungsi ini menghasilkan prediksi untuk setiap sampel uji dan memberikan hasil akhir dalam bentuk array.

Gaussian Naive Bayes

Untuk mengimplementasikan algoritma ini, pertama, hitung peluang awal atau *probabilitas prior* untuk setiap kategori yang dilakukan dengan membagi jumlah data dalam suatu kategori dengan total seluruh data. Kedua, hitung peluang suatu fitur muncul dalam kategori tertentu (*probabilitas likelihood*). Jika fitur *numerical*, dapat digunakan cara khusus untuk memeriksa bagaimana angka-angka tersebut menyebar di kategori tersebut. Ketiga, kita menggabungkan semua peluang ini untuk setiap kategori, dengan mengalikan peluang awal dan peluang masing-masing fitur. Terakhir, kategori dengan peluang tertinggi akan dipilih sebagai hasilnya.



```python
import numpy as np
import pandas as pd

class GaussianNaiveBayes:
    def __init__(self, var_smoothing=1e-9):
        self.classes_ = None
        self.class_priors_ = None
        self.theta_ = None
        self.var_ = None
        self.var_smoothing = var_smoothing

    def _validate_and_convert(self, X, y=None):
        """
        Validates and converts the input data (X and y) to a compatible format.
        Ensures X is numeric and y is of valid type and shape.
        """
        # Validate and convert X
        if not isinstance(X, (pd.DataFrame, np.ndarray)):
            raise ValueError("X must be a pandas DataFrame or a NumPy array.")
        if isinstance(X, pd.DataFrame):
            X = X.copy()
            X = X.apply(pd.to_numeric, errors='coerce')  # Ensure all values are numeric
        X = np.array(X, dtype=np.float64)

        if np.any(np.isnan(X)):
            raise ValueError("X contains NaN values. Handle missing values before passing data.")

        # Validate and convert y
        if y is not None:
            if not isinstance(y, (pd.Series, np.ndarray, list)):
                raise ValueError("y must be a pandas Series, NumPy array, or list.")
            y = np.array(y, dtype=np.float64)

            if len(y) != len(X):
                raise ValueError("X and y must have the same number of samples.")

        return X, y
```

```python
    def fit(self, X, y):
        """
        Fits the Gaussian Naive Bayes model with the training data.
        Validates and converts the input data.
        """

        # Validate and convert inputs
        X, y = self._validate_and_convert(X, y)

        # Identify unique classes and their counts
        self.classes_, class_counts = np.unique(y, return_counts=True)

        # Compute class priors (P(Ck))
        self.class_priors_ = class_counts / y.shape[0]

        # Compute mean (theta) and variance (var) for each class and feature
        self.theta_ = np.array([X[y == c].mean(axis=0) for c in self.classes_])
        self.var_ = np.array([X[y == c].var(axis=0, ddof=1) for c in self.classes_])

        # Add a small smoothing factor to variances for numerical stability
        self.var_ += self.var_smoothing * np.var(X, axis=0).max()

    def _log_likelihood(self, X):
        """
        Computes the log-likelihood of the input data for each class.
        """

        log_likelihoods = []

        for i, _ in enumerate(self.classes_):
            # Calculate log likelihood for each class
            mean = self.theta_[i]
            variance = self.var_[i]

            # Avoid large exponentiation by computing in log space
            log_prob = -0.5 * (np.sum(np.log(2 * np.pi * variance)) + np.sum(((X - mean) ** 2) / variance, axis=1))

            log_likelihoods.append(log_prob)

        return np.array(log_likelihoods).T

    def predict(self, X):
        """
        Predicts the class labels for the input data.
        Validates and converts the input data.
        """

        # Validate and convert X
        X, _ = self._validate_and_convert(X)

        # Compute log prior probabilities
        log_priors = np.log(self.class_priors_)

        # Compute log-likelihood for each class
        log_likelihoods = self._log_likelihood(X)

        # Combine priors and likelihoods to compute posterior
        log_posteriors = log_likelihoods + log_priors

        # Return class with highest posterior probability
        return self.classes_[np.argmax(log_posteriors, axis=1)]

```

Gambar 6.2 Model implementation Gaussian Naive Bayes

Sama seperti pada algoritma KNN, algoritma GNB ini juga diimplementasikan sebagai sebuah class dimana di dalam class ini terdapat 5 buah fungsi, yaitu.

1. __init__

   Fungsi ini adalah konstruktor untuk menginisialisasi objek kelas GaussianNaiveBayes. Parameter var_smoothing digunakan untuk menambahkan nilai kecil pada varians selama proses perhitungan untuk mencegah pembagian dengan nol. Konstruktor ini juga mendeklarasikan atribut-atribut seperti classes_, class_priors_, theta_, dan var_, yang akan diisi selama pelatihan model.

2. _validate_and_convert

   Metode ini bertugas untuk memvalidasi dan mengonversi data masukan X (fitur) dan y (label) agar sesuai dengan format yang diharapkan. Data X dikonversi menjadi array numerik (jika berupa DataFrame), dan dicek apakah ada nilai NaN. Untuk y, metode ini memastikan bahwa jumlah sampel sesuai dengan X, dan tipe datanya valid (list, NumPy array, atau pandas Series). Metode ini membantu memastikan integritas data sebelum diproses lebih lanjut.

3. fit

   Metode ini melatih model Gaussian Naive Bayes menggunakan data pelatihan X dan y. Setelah memvalidasi data dengan memanggil _validate_and_convert, metode ini menghitung:

   - classes_: Label unik di y.
   - class_priors_: Probabilitas awal dari masing-masing kelas (P(Ck)).
   - theta_: Mean (rata-rata) fitur untuk setiap kelas.
   - var_: Varians fitur untuk setiap kelas. Nilai varians juga diberikan smoothing untuk meningkatkan stabilitas numerik.

4. _log_likelihood

   Metode ini menghitung log-likelihood (logaritma probabilitas data diberikan kelas tertentu) untuk setiap kelas dalam X. Dengan memanfaatkan mean dan varians dari masing-masing kelas yang dihitung saat pelatihan, metode ini menerapkan formula distribusi Gaussian multivariat dalam bentuk logaritmik. Hal ini mencegah masalah underflow saat mengalikan probabilitas kecil.

5. predict

   Metode ini digunakan untuk memprediksi label kelas untuk data masukan X. Pertama, data X divalidasi menggunakan _validate_and_convert. Kemudian:

   - Log-prior dihitung dari probabilitas awal kelas (class_priors_).
   - Log-likelihood dihitung menggunakan _log_likelihood.
   - Posterior dalam log-space diperoleh dengan menjumlahkan log-prior dan log-likelihood. Akhirnya, kelas dengan probabilitas posterior tertinggi dipilih sebagai prediksi untuk setiap sampel.

HASIL DAN ANALISIS

Setelah data dipersiapkan dan juga model dibuat maka langkah selanjutnya yaitu mengevaluasi performa model. Evaluasi model ini menggunakan teknik split 80:20 dimana 80% data akan menjadi data train dan 20% data akan menjadi data validasi. Disini kami menggunakan 2 model yaitu KNN dan juga Gaussian Naive Bayes dengan 2 pendekatan yaitu implementasi from scratch dan juga menggunakan library sklearn sebagai pembanding. Untuk metrik utama yang dipakai itu F1-Score macro dimana menyesuaikan dengan isu imbalance data. Hasilnya sebagai berikut.

```
KNN Accuracy: 0.994693921156654
KNN ROC AUC: 0.9694291539048369
KNN Recall: 0.9694291539048367
KNN Precision: 0.9928492591580735
KNN F1 Score: 0.980798744992893
              precision    recall  f1-score   support

           0       0.99      0.94      0.96      2152
           1       1.00      1.00      1.00     25929

    accuracy                           0.99     28081
   macro avg       0.99      0.97      0.98     28081
weighted avg       0.99      0.99      0.99     28081
```

Gambar 7.1 Hasil KNN from scratch


```
KNN Accuracy: 0.994693921156654
KNN ROC AUC: 0.9694291539048369
KNN Recall: 0.9694291539048367
KNN Precision: 0.9928492591580735
KNN F1 Score: 0.980798744992893
              precision    recall  f1-score   support

           0       0.99      0.94      0.96      2152
           1       1.00      1.00      1.00     25929

    accuracy                           0.99     28081
   macro avg       0.99      0.97      0.98     28081
weighted avg       0.99      0.99      0.99     28081
```


Gambar 7.2 Hasil KNN Sklearn

```
GNB Accuracy: 0.9806986930664863
GNB ROC AUC: 0.8779056864032909
GNB Recall: 0.8779056864032909
GNB Precision: 0.9846210965662124
GNB F1 Score: 0.9234713448522472
              precision    recall  f1-score   support

           0       0.99      0.76      0.86      2152
           1       0.98      1.00      0.99     25929

    accuracy                           0.98     28081
   macro avg       0.98      0.88      0.92     28081
weighted avg       0.98      0.98      0.98     28081
```

Gambar 7.3 Hasil GNB from scratch


```
GNB Accuracy: 0.9806986930664863
GNB ROC AUC: 0.8779056864032909
GNB Recall: 0.8779056864032909
GNB Precision: 0.9846210965662124
GNB F1 Score: 0.9234713448522472
              precision    recall  f1-score   support

           0       0.99      0.76      0.86      2152
           1       0.98      1.00      0.99     25929

    accuracy                           0.98     28081
   macro avg       0.98      0.88      0.92     28081
weighted avg       0.98      0.98      0.98     28081
```


Gambar 7.4 Hasil GNB Sklearn

Perbandingan hasil scikit-learn dan KNN

Implementasi KNN baik dari scratch maupun menggunakan Sklearn menunjukkan hasil evaluasi yang identik, dengan accuracy (99.46%), ROC AUC (0.969), recall (0.969), precision (0.992), dan F1-score (0.980). Kesamaan hasil ini mengindikasikan bahwa implementasi dari scratch telah berhasil mereplikasi cara kerja Sklearn secara akurat. KNN bekerja dengan menghitung jarak antar data menggunakan metrik seperti Euclidean atau Manhattan dan menentukan kelas berdasarkan mayoritas tetangga terdekat (n_neighbors). Keberhasilan ini juga didukung oleh parameter yang sama antara kedua metode. Tidak ada perbedaan signifikan, yang menunjukkan bahwa prinsip dasar algoritma KNN telah diterapkan dengan benar.

Perbandingan hasil hasil scikit-learn dan Naive-Bayes

Hasil evaluasi Gaussian Naive Bayes (GNB) dari scratch dan Sklearn juga identik, dengan accuracy (98.08%), ROC AUC (0.877), recall (0.877), precision (0.984), dan F1-score (0.923). Namun, performa GNB lebih rendah dibandingkan KNN, terutama pada label minoritas (kelas 0), yang terlihat dari recall (0.76) untuk label tersebut. Hal ini menunjukkan bahwa model sering salah memprediksi kelas minoritas sebagai kelas mayoritas. Penyebab utama kemungkinan adalah asumsi distribusi normal yang digunakan GNB tidak sepenuhnya sesuai dengan karakteristik data, serta distribusi kelas yang tidak seimbang (class imbalance). Meskipun begitu, hasil yang identik antara implementasi dari scratch dan Sklearn menunjukkan bahwa perhitungan mean, varians, dan probabilitas posterior telah diterapkan dengan benar.

Perbandingan hasil hasil KNN dan GNB

Jika dibandingkan, KNN secara signifikan lebih unggul daripada GNB dalam menangani dataset ini. Hal ini terlihat dari macro average F1-score yang lebih tinggi untuk KNN (0.98 vs. 0.92). KNN menunjukkan performa yang lebih konsisten di semua kelas, terutama karena metodenya tidak bergantung pada asumsi distribusi data, melainkan pada kedekatan jarak antar data. Sebaliknya, GNB lebih cocok untuk dataset dengan distribusi normal yang seimbang, sehingga kurang efektif untuk dataset dengan distribusi kelas yang tidak seimbang seperti ini. KNN juga mampu menangkap pola yang lebih kompleks, meskipun membutuhkan lebih banyak sumber daya komputasi dibandingkan GNB. Secara keseluruhan, KNN adalah pilihan yang lebih baik untuk dataset ini karena fleksibilitasnya dan kemampuannya dalam menangani ketidakseimbangan kelas, sedangkan GNB lebih terbatas oleh asumsi distribusinya.

HYPERPARAMETER TUNING

Sebuah model itu memiliki parameter yang dapat dioptimalkan, karena itu diperlukan tuning yang bertujuan untuk mencari kombinasi parameter terbaik. Salah satu cara yang dapat dilakukan yaitu dengan menggunakan GridSearchCV dimana seluruh kombinasi parameter yang telah ditentukan sebelumnya dicoba dengan metrik yang kali ini digunakan yaitu F1 Score Macro. Selain itu dalam tune ini digunakan cross validation sebanyak 5. Berikut merupakan hasilnya.

```
Fitting 5 folds for each of 30 candidates, totalling 150 fits
Best Score: 0.9841
Best Parameters: {'metric': 'manhattan', 'n_neighbors': 6}
```

Gambar 8.1 KNN Parameter Tuned

```
Best parameters: {'var_smoothing': 0.001}
Best F1-Score: 0.9781018412192266
```

Gambar 8.2 GNB Parameter Tuned

```
KNN Accuracy: 0.994693921156654
KNN ROC AUC: 0.9694291539048369
KNN Recall: 0.9694291539048367
KNN Precision: 0.9928492591580735
KNN F1 Score: 0.980798744992893
              precision    recall  f1-score   support

           0       0.99      0.94      0.96      2152
           1       1.00      1.00      1.00     25929

    accuracy                           0.99     28081
   macro avg       0.99      0.97      0.98     28081
weighted avg       0.99      0.99      0.99     28081
```

Gambar 8.3 KNN Base Model

```
KNN Accuracy: 0.9945870873544389
KNN ROC AUC: 0.9693713036213704
KNN Recall: 0.9693713036213705
KNN Precision: 0.9921219479399592
KNN F1 Score: 0.9804250172080322
              precision    recall  f1-score   support

           0       0.99      0.94      0.96      2152
           1       1.00      1.00      1.00     25929

    accuracy                           0.99     28081
   macro avg       0.99      0.97      0.98     28081
weighted avg       0.99      0.99      0.99     28081
```

Gambar 8.4 KNN Tuned

```
GNB Accuracy: 0.9806986930664863
GNB ROC AUC: 0.8779056864032909
GNB Recall: 0.8779056864032909
GNB Precision: 0.9846210965662124
GNB F1 Score: 0.9234713448522472
              precision    recall  f1-score   support

           0       0.99      0.76      0.86      2152
           1       0.98      1.00      0.99     25929

    accuracy                           0.98     28081
   macro avg       0.98      0.88      0.92     28081
weighted avg       0.98      0.98      0.98     28081
```

Gambar 8.5 GNB Base Model

```
GNB Accuracy: 0.9932694704604537
GNB ROC AUC: 0.9633313523016311
GNB Recall: 0.9633313523016312
GNB Precision: 0.9886546910601606
GNB F1 Score: 0.9755905165493355
              precision    recall  f1-score   support

           0       0.98      0.93      0.95      2152
           1       0.99      1.00      1.00     25929

    accuracy                           0.99     28081
   macro avg       0.99      0.96      0.98     28081
weighted avg       0.99      0.99      0.99     28081
```

Gambar 8.6 GNB Tuned

Perbandingan hasil KNN Base Model vs Tuned

Pada base model KNN, performa menunjukkan hasil yang sudah sangat baik dengan accuracy (99.46%), F1-score (0.980), dan ROC AUC (0.969). Model menggunakan parameter default tanpa optimasi, yaitu metrik jarak "Euclidean" dan jumlah tetangga k = 3. Setelah dilakukan hyperparameter tuning menggunakan GridSearchCV, parameter terbaik yang ditemukan adalah metrik "Manhattan" dengan n_neighbors = 6, menghasilkan F1-score (0.984) yang sedikit lebih baik dari base model. Peningkatan ini menunjukkan bahwa dengan metrik jarak yang lebih sensitif terhadap dimensi tertentu (Manhattan), model mampu meningkatkan prediksi, khususnya pada distribusi data

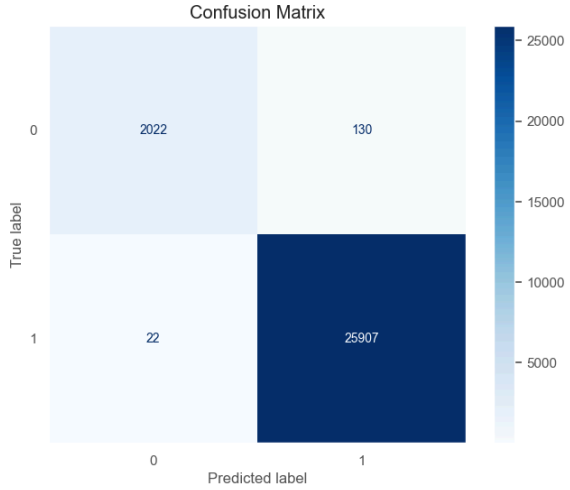
yang tidak sepenuhnya simetris. Namun, peningkatan performa relatif kecil karena base model sudah cukup optimal.

Perbandingan hasil GNB Base Model vs Tuned

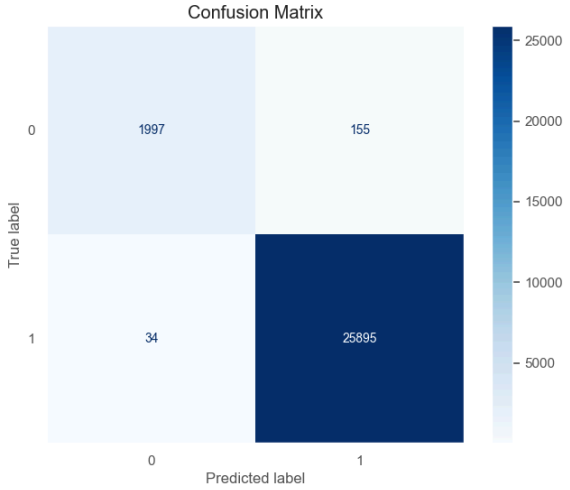
Base model Gaussian Naive Bayes menggunakan parameter default dengan var_smoothing = 1e-9, menghasilkan accuracy (98.08%), F1-score (0.923), dan ROC AUC (0.877). Setelah tuning dengan variasi var_smoothing menggunakan GridSearchCV, nilai terbaik ditemukan pada var_smoothing = 0.001, menghasilkan F1-score (0.975) yang signifikan lebih tinggi dari base model. Peningkatan ini mengindikasikan bahwa regularisasi yang lebih besar (var_smoothing lebih tinggi) membantu mengatasi ketidakseimbangan kelas dan mengurangi sensitivitas model terhadap outlier atau variabilitas data yang tinggi.

ERROR ANALYSIS

Error analysis merupakan langkah penting dalam evaluasi model machine learning untuk memahami sejauh mana model dapat memprediksi data secara akurat dan mengidentifikasi potensi kelemahan yang perlu diperbaiki. Untuk error analysis ini akan digunakan confussion matrix yaitu sebuah matrix yang dapat menunjukan bagaimana perbandingan antara true label dan predicted label. Berikut merupakan hasilnya.



Gambar 9.1 Confussion Matrix KNN Tuned Model



Gambar 9.2 Confussion Matrix GNB Tuned Model

KNN Tuned Model

- **True Positive (TP):** Sebanyak 25,907 sampel kelas positif (1) berhasil diprediksi dengan benar. Ini menunjukkan bahwa model KNN Tuned memiliki sensitivitas yang sangat tinggi terhadap kelas positif. TP yang tinggi menunjukkan kemampuan model untuk mendeteksi kasus yang benar-benar relevan, yang sering kali menjadi prioritas dalam kasus klasifikasi.
- **True Negative (TN):** Sebanyak 2,022 sampel kelas negatif (0) berhasil diprediksi dengan benar. Angka TN ini menunjukkan bahwa model juga cukup andal dalam mengenali kelas negatif.
- **False Positive (FP):** Sebanyak 130 sampel kelas negatif (0) salah diprediksi sebagai kelas positif (1). FP yang rendah menunjukkan bahwa model KNN Tuned memiliki tingkat kesalahan yang kecil dalam mendeteksi kelas negatif. Hal ini relevan dalam konteks di mana kesalahan FP dapat menyebabkan dampak negatif, seperti false alarm.
- **False Negative (FN):** Sebanyak 22 sampel kelas positif (1) salah diprediksi sebagai kelas negatif (0). FN yang sangat rendah menegaskan bahwa model ini hampir selalu dapat mendeteksi kelas positif dengan benar, yang sangat penting jika kelas positif adalah fokus utama.

GNB Tuned Model

- **True Positive (TP):** Sebanyak 25,895 sampel kelas positif (1) berhasil diprediksi dengan benar. Meskipun sedikit lebih rendah dibandingkan KNN Tuned, TP ini tetap menunjukkan performa yang sangat baik dalam mendeteksi kelas positif.
- **True Negative (TN):** Sebanyak 1,997 sampel kelas negatif (0) berhasil diprediksi dengan benar. TN ini lebih rendah dibandingkan KNN Tuned, menunjukkan bahwa model GNB sedikit kurang akurat dalam mengenali kelas negatif.
- **False Positive (FP):** Sebanyak 155 sampel kelas negatif (0) salah diprediksi sebagai kelas positif (1). FP yang sedikit lebih tinggi dibandingkan KNN menunjukkan bahwa GNB cenderung lebih banyak membuat false alarm, yang dapat menjadi perhatian jika FP memiliki konsekuensi yang signifikan.
- **False Negative (FN):** Sebanyak 34 sampel kelas positif (1) salah diprediksi sebagai kelas negatif (0). FN yang lebih tinggi dibandingkan KNN Tuned menunjukkan bahwa GNB memiliki sedikit kelemahan dalam mendeteksi semua kasus kelas positif dengan benar.

PEMBAGIAN TUGAS

Nama	NIM	Pembagian Tugas
Dinda Thalia Fahira	18222055	GNB, Notebook, Laporan
Mohamad Maulana Firdaus R	18222140	KNN, Notebook, Laporan

REFERENSI

Bhavya22, & Follow. (2024, July 10). *ML: Naive Bayes scratch implementation using Python*. GeeksforGeeks. https://www.geeksforgeeks.org/ml-naive-bayes-scratch-implementation-using-python/

Kartik, & Follow. (2024, July 15). *K-Nearest Neighbor(KNN) algorithm*. GeeksforGeeks. https://www.geeksforgeeks.org/k-nearest-neighbours/